

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №6

“Объектно-ориентированные возможности языка C#”

Выполнил:

Студент группы ИУ5-35Б

Клокова М.А.

Преподаватель:

Гапанюк Ю.Е.

Москва 2025

Постановка задачи:

1. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке C#.
2. Все файлы проекта (кроме основного файла main.cs) должны располагаться в пакете labOop.
3. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета labOop.
4. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры.
5. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры.
6. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.
7. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус».
8. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:
 - a. Определите метод "get", который возвращает в виде строки основные параметры фигуры, ее цвет и площадь.
 - b. Название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.
9. В корневом каталоге проекта создайте файл main.py для тестирования ваших классов. Создайте следующие объекты и выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):
 - a. Прямоугольник синего цвета шириной N и высотой N.
 - b. Круг зеленого цвета радиусом N.
 - c. Квадрат красного цвета со стороной N.

Код программы:

main.cs:

```
using System;
using System.Collections.Generic;

namespace BiquadraticEquation {
    class Program {
        static double GetCoefficient(int index, string prompt, string[] args) {
            try {
                if (index < args.Length) {
                    return Convert.ToDouble(args[index]);
                }
            }
            catch (IndexOutOfRangeException) { }
            catch (FormatException) { }

            Console.Write(prompt);
            string coefStr = Console.ReadLine();
            return Convert.ToDouble(coefStr);
        }

        static List<double> GetRoots(double a, double b, double c) {
            List<double> result = new List<double>();

            double D = b * b - 4 * a * c;

            // для сравнения дробей
            const double eps = 1e-12;

            if (Math.Abs(D) < eps) {
                double y = -b / (2.0 * a);
                if (Math.Abs(y) < eps) {
                    result.Add(0.0);
                }
                else if (y > 0.0) {
                    double root = Math.Sqrt(y);
                    result.Add(root);
                    result.Add(-root);
                }
            }
            else if (D > 0.0) {
                double sqrtD = Math.Sqrt(D);
                double y1 = (-b + sqrtD) / (2.0 * a);
                double y2 = (-b - sqrtD) / (2.0 * a);

                if (Math.Abs(y1) < eps) { //
                    result.Add(0.0);
                }
                else if (y1 > 0.0) {
                    double root1 = Math.Sqrt(y1);
```

```

        result.Add(root1);
        result.Add(-root1);
    }

    if (Math.Abs(y2) < eps) { //
        if (!result.Contains(0.0))
        {
            result.Add(0.0);
        }
    }
    else if (y2 > 0.0) {
        double root2 = Math.Sqrt(y2);
        result.Add(root2);
        result.Add(-root2);
    }
}

return result;
}

static void Main(string[] args) {
    Console.OutputEncoding = System.Text.Encoding.UTF8;

    double a = GetCoefficient(0, "Введите коэффициент A: ", args);
    double b = GetCoefficient(1, "Введите коэффициент B: ", args);
    double c = GetCoefficient(2, "Введите коэффициент C: ", args);

    List<double> roots = GetRoots(a, b, c);

    roots.Sort();

    int lenRoots = roots.Count;
    if (lenRoots == 0) {
        Console.WriteLine("Нет действительных корней");
    }
    else if (lenRoots == 1) {
        Console.WriteLine($"Один корень: {roots[0]}");
    }
    else if (lenRoots == 2) {
        Console.WriteLine($"Два корня: {roots[0]} и {roots[1]}");
    }
    else if (lenRoots == 3) {
        Console.WriteLine($"Три корня: {roots[0]}, {roots[1]} и
{roots[2]}");
    }
    else if (lenRoots == 4) {
        Console.WriteLine($"Четыре корня: {roots[0]}, {roots[1]}, {roots[2]}
и {roots[3]}");
    }
    Console.ReadKey();
}

```

```
    }  
}
```

LabOop/GeometricFigure.cs:

```
using System;  
  
namespace lab_6.LabOop {  
    public abstract class GeometricFigure {  
        public virtual string FigureName { get; } = "Геометрическая фигура";  
  
        public abstract double CalculateArea();  
  
        public abstract string Repr();  
    }  
}
```

LabOop/FigureColor.cs:

```
namespace lab_6.LabOop {  
    public class Color {  
        private string color;  
        public Color(string color = "") {  
            this.color = color;  
        }  
  
        public string ColorName {  
            get { return color; }  
            set { color = value; }  
        }  
        public override string ToString() {  
            return color;  
        }  
    }  
}
```

LabOop /Rectangle.cs:

```
using System;
```

```
namespace lab_6.LabOop {
    public class Rectangle : GeometricFigure {
        private double width;
        private double height;
        private Color color;

        public override string FigureName { get; } = "Прямоугольник";

        public Rectangle(double width, double height, string color) {
            if (width <= 0 || height <= 0)
                throw new ArgumentException("Ширина и высота должны быть положительными числами");

            this.width = width;
            this.height = height;
            this.color = new Color(color);
        }

        public double Width {
            get { return width; }
        }

        public double Height {
            get { return height; }
        }

        public string FigureColor {
            get { return color.ColorName; }
        }

        public override double CalculateArea() {
            return width * height;
        }

        public override string Repr() {
            return $"Фигура: {FigureName}, ширина: {width:F2}, высота: {height:F2}, цвет: {color}, площадь: {CalculateArea():F2}";
        }
    }
}
```

```

        public override string ToString() {
            return Repr();
        }
    }
}

```

LabOop /Circle.cs:

```

using System;

namespace lab_6.LabOop {
    public class Circle : GeometricFigure {
        private double radius;
        private Color color;

        public override string FigureName { get; } = "Круг";

        private const double PI = Math.PI;

        public Circle(double radius, string color) {
            if (radius <= 0)
                throw new ArgumentException("Радиус должен быть положительным числом");

            this.radius = radius;
            this.color = new Color(color);
        }

        public double Radius {
            get { return radius; }
        }

        public string FigureColor {
            get { return color.ColorName; }
        }

        public override double CalculateArea() {
            return PI * radius * radius;
        }
    }
}

```

```

        public override string Repr() {
            return $"Фигура: {FigureName}, радиус: {radius:F2}, цвет: {color},
площадь: {CalculateArea():F2}";
        }

        public override string ToString() {
            return Repr();
        }
    }
}

```

LabOop /Square.py:

```

using System;

namespace lab_6.LabOop {
    public class Square : Rectangle {
        public override string FigureName { get; } = "Квадрат";

        public Square(double side, string color) : base(side, side, color) {
        }

        public double Side {
            get { return Width; }
        }

        public override string Repr() {
            return $"Фигура: {FigureName}, сторона: {Side:F2}, цвет: {FigureColor},
площадь: {CalculateArea():F2}";
        }

        public override string ToString() {
            return Repr();
        }
    }
}

```


Результаты выполнения программы:

- afeelu@mbp-stanislav lab_2 % python3 main.py
Фигура: Прямоугольник, ширина: 11, высота: 11, цвет: синий, площадь: 121
Фигура: Круг, радиус: 11, цвет: зеленый, площадь: 380.13
Фигура: Квадрат, сторона: 11, цвет: красный, площадь: 121

Все фигуры:
 - Прямоугольник: площадь = 121.00
 - Круг: площадь = 380.13
 - Квадрат: площадь = 121.00
- afeelu@mbp-stanislav lab 2 % █